SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

Engineer to Excel

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures COURSE CODE: CSA03

Register Number	**19231041**
Name	**P.NANDHINI**

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2 **EACH QUESTIONS CARRIES:** 10 MARKS **Total Marks:**20 **Duration:** ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I am nandhini (192313041) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary

action. ***Signature of the student:P.Nandhini***



RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Concept Mapping Assignment

Data Structures in Healthcare and Smart City Systems

Q1. Smart Healthcare System: Data to Implementation in C

A smart healthcare system processes large volumes of patient data such as temperature readings, medical history and diagnostic details. The concept map below traces how this data progresses from raw facts to a working C implementation. The main progression runs down the left column. Sub-concepts at each level are shown on the right.

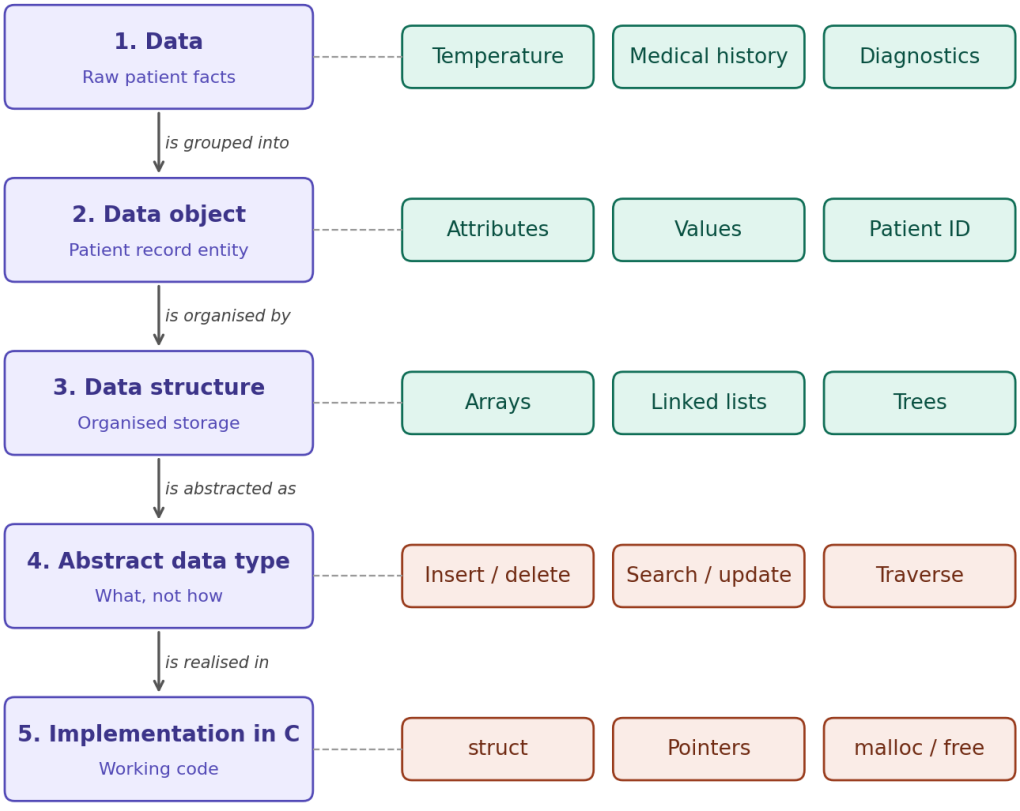


Figure 1: Hierarchical concept map from Data to Implementation in C

Linkages between concepts

- **Data to Data object:** individual readings (temperature, history, diagnostics) are grouped into a single entity, the patient record, made up of attributes and values.
- **Data object to Data structure:** many patient records must be stored and organised in memory using arrays, linked lists or trees.

- **Data structure to ADT:** the structure is wrapped by an abstract data type that defines operations (insert, delete, search, update, traverse) without exposing how they work.
- **ADT to Implementation in C:** the ADT is realised with a struct for the record, pointers to link records, and functions that carry out each operation, with malloc and free managing memory.

Justification of each level

Level	Contribution to efficient organization, processing and performance
1. Data	Holds the raw patient facts. On its own it is unorganised and cannot be searched or processed efficiently.
2. Data object	Groups related data into one meaningful entity. A unique Patient ID lets the system locate a record directly instead of scanning everything.
3. Data structure	Decides how records are arranged in memory. Arrays give fast indexed access, linked lists make insertion and deletion cheap, and trees keep data ordered for fast search.
4. Abstract data type	Separates what the system does from how it does it. Doctors and modules use the operations only, so storage can be changed or optimised without breaking the rest of the system.
5. Implementation in C	Turns the design into working code. struct bundles patient attributes, pointers link records dynamically, and malloc and free let the system grow and shrink with patient load.

Effect on performance: matching the structure to the dominant operation improves speed. Array lookup by index is O(1), linked-list insertion is O(1) once the position is known, and search in a balanced tree is O(log n). Choosing well reduces processing time as patient volume grows.

Q2. Smart City Traffic Management: Classification of Data Structures

A smart city traffic management system handles data about vehicle movement, road networks and signal control. The concept map below classifies data structures from primitive to non-primitive and from linear to non-linear, with a traffic application for each.

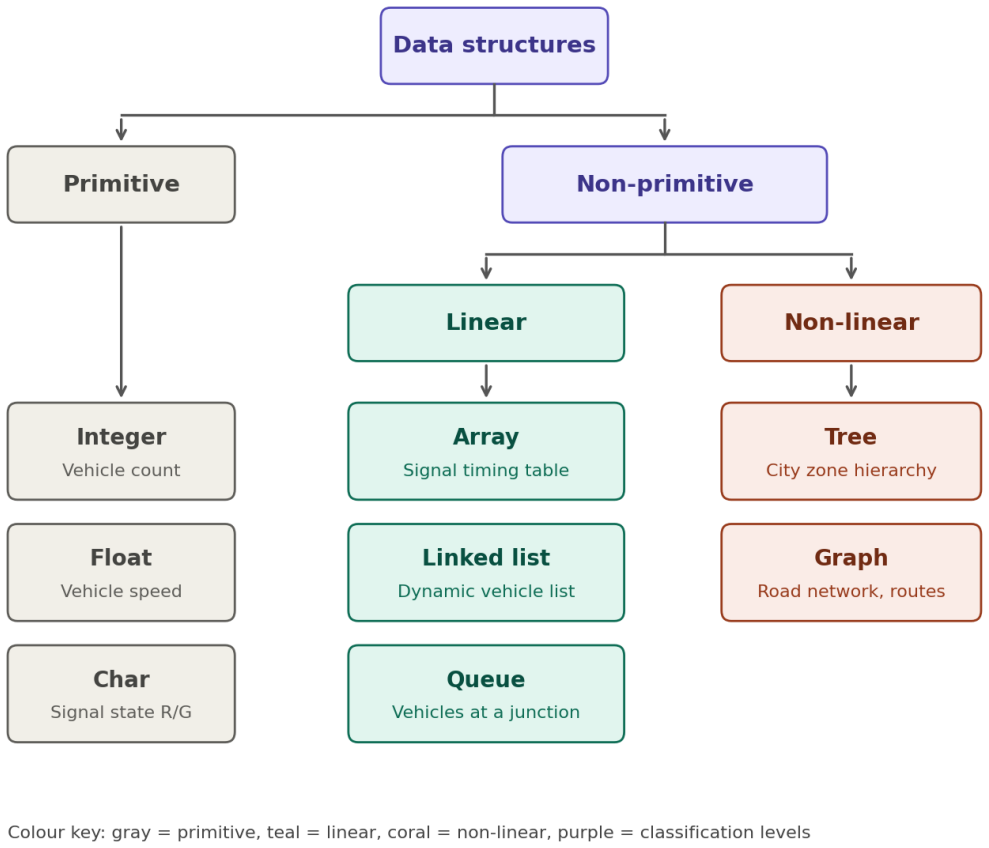


Figure 2: Classification of data structures with traffic management examples

Relationships between the structures

- **Primitive to non-primitive:** primitive types (int, float, char) hold a single value and are the building blocks. Non-primitive structures are built from them to hold collections. A vehicle record, for example, combines an integer ID, a float speed and a char signal state.
- **Linear to non-linear:** in linear structures each element has one predecessor and one successor, so data forms a sequence. In non-linear structures an element can connect to many others, forming a hierarchy or a network.
- **Between structures:** queues and stacks are linear structures that can be built on arrays or linked lists. A tree is a linked structure in which each node has several child links. A graph is the most general form, and a tree is a graph with no cycles.

Applications in traffic management

Structure	Traffic application	Why it fits
Array	Signal timing table for each junction	Fixed size with O(1) index access. Fast, but resizing is inflexible.

Structure	Traffic application	Why it fits
Linked list	Dynamic list of vehicles on a road	Cheap insertion and deletion as vehicles enter and leave. Access is O(n).
Queue (FIFO)	Vehicles waiting at a signal	Matches first-come, first-served behaviour.
Tree	City, zone and junction hierarchy, or sorted records for fast lookup	Models hierarchical data. Search is O(log n) when balanced.
Graph	Roads as edges and junctions as nodes, for shortest-path routing and congestion analysis	The only structure that models many-to-many connections between locations.

Justification of the classification

- **Functionality:** linear structures represent one-to-one sequences such as queues of vehicles or timing tables. Non-linear structures represent one-to-many (tree) and many-to-many (graph) relationships such as zones and road networks.
- **Efficiency:** linear structures are simple and memory-light, but searching them is O(n). Non-linear structures use more memory and are more complex, but make hierarchical and network operations far faster.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.

Tenenbaum, A. M., Langsam, Y., & Augenstein, M. J. Data Structures Using C. Pearson.

Weiss, M. A. Data Structures and Algorithm Analysis in C. Pearson.

Note: add the edition and year of the copies you used, or replace these with your prescribed course textbook.